

# PREPARATION REPORT LAB3

## ADVANCED CPU ARCHITECTURE AND HARDWARE

### ACCELERATORS LAB

361.1.4693

Yoav Tsur 211903406

Elad Lavi 211626551

2026 Semester B

## מבוא

במעבדה זו מימשנו מעבד רב-מחזורי בשפת VHDL, המבוסס על הפרדה בין יחידת בקרה (Control Unit) לבין מסלול נתונים (Datapath). מטרת המעבדה הייתה לתכנן מערכת המסוגלת להריץ קוד Assembly נתון, לבצע פקודות אריתמטיות ולוגיות, גישות לזיכרון, ו-פקודות קפיצה מותנות ולא מותנות.

המערכת בנויה ממספר רכיבים עיקריים: יחידת בקרה המבוססת על FSM, מסלול נתונים הכולל ALU, Register File, Program Counter, Instruction Register, וזיכרונות עבור פקודות ונתונים. יחידת הבקרה מקבלת מה-Datapath חיווי לגבי סוג הפקודה והדגלים הרלוונטיים, ובהתאם לכך מפיקה את אותות הבקרה הדרושים לביצוע כל שלב במחזור הפקודה.

בנוסף למימוש המעבד, נדרשנו לבצע סימולציה מבוססת קבצים. בתחילת הסימולציה ה-Testbench טוען את תוכן זיכרון הפקודות ואת תוכן זיכרון הנתונים מקבצים בינאריים שנוצרו מקוד אסמבלי. לאחר סיום ריצת התוכנית, כאשר אות ה-done עולה, ה-Testbench קורא את תוכן זיכרון הנתונים וכותב אותו לקובץ פלט. קובץ זה משמש להשוואה מול הפלט הצפוי.

במסגרת העבודה נבדקה תקינות המערכת באמצעות מספר תוכניות Assembly שונות, שכל אחת מהן בוחנת פעולות שונות של המעבד, כגון חיבור, חיסור, פעולות לוגיות, גישה לזיכרון, וקפיצות מותנות. תהליך האימות בוצע באמצעות ModelSim, תוך שימוש בגלים, מעקב אחר מצבי ה-FSM, בדיקת ערכי רגיסטרים וזיכרון.

בנוסף להרצת תוכניות בדיקה שסופקו במסגרת המעבדה, חלק מהמשימות דרשו כתיבה ופיתוח של תוכניות אסמבלי ייעודיות לצורך בדיקת תקינות המעבד ומימוש פקודות שונות.

המעבד שמומש בפרויקט זה מבוסס על ארכיטקטורת סט הפקודות (ISA) שהוגדרה במסגרת המעבדה. הטבלה הבאה מציגה את הפקודות הנתמכות, פורמט הפקודות, ואופן פעולתן.

| Instruction Format | Decimal value | OPC  | Instruction          | Explanation                                                   | N | Z | C |
|--------------------|---------------|------|----------------------|---------------------------------------------------------------|---|---|---|
| R-Type             | 0             | 0000 | add ra,rb,rc         | $R[ra] \leftarrow R[rb] + R[rc]$                              | * | * | * |
|                    |               |      | nop                  | $R[0] \leftarrow R[0] + R[0]$ ( <i>emulated instruction</i> ) | * | * | * |
|                    | 1             | 0001 | sub ra,rb,rc         | $R[ra] \leftarrow R[rb] - R[rc]$                              | * | * | * |
|                    | 2             | 0010 | and ra,rb,rc         | $R[ra] \leftarrow R[rb] \text{ and } R[rc]$                   | * | * | * |
|                    | 3             | 0011 | or ra,rb,rc          | $R[ra] \leftarrow R[rb] \text{ or } R[rc]$                    | * | * | * |
|                    | 4             | 0100 | xor ra,rb,rc         | $R[ra] \leftarrow R[rb] \text{ xor } R[rc]$                   | * | * | * |
|                    | 5             | 0101 | unused               |                                                               |   |   |   |
| J-Type             | 6             | 0110 | unused               |                                                               |   |   |   |
|                    | 7             | 0111 | jmp offset_addr      | $PC \leftarrow PC + 1 + \text{offset\_addr}$                  | - | - | - |
|                    | 8             | 1000 | jc /jhs offset_addr  | If(Cflag==1) $PC \leftarrow PC + 1 + \text{offset\_addr}$     | - | - | - |
|                    | 9             | 1001 | jnc /jlo offset_addr | If(Cflag==0) $PC \leftarrow PC + 1 + \text{offset\_addr}$     | - | - | - |
|                    | 10            | 1010 | unused               |                                                               |   |   |   |
| I-Type             | 11            | 1011 | unused               |                                                               |   |   |   |
|                    | 12            | 1100 | mov ra,imm           | $R[ra] \leftarrow \text{imm}$                                 | - | - | - |
|                    | 13            | 1101 | ld ra,imm(rb)        | $R[ra] \leftarrow M[\text{imm} + R[rb]]$                      | - | - | - |
| Special            | 14            | 1110 | st ra,imm(rb)        | $M[\text{imm} + R[rb]] \leftarrow R[ra]$                      | - | - | - |
|                    | 15            | 1111 | done                 | Signals the TB to read the DTCM content                       | - | - | - |

Note: \* The status flag bit is affected , - The status flag bit is not affected

מימוש יחידת הבקרה התבצע באמצעות FSM מסוג Mealy, יחידה זו אחראית על הפקת אותות הבקרה בהתאם למצב הנוכחי של המכונה ולקלטים הרלוונטיים, כגון סוג הפקודה ודגלי המצב.

יחידת ה-Datapath ממומשת בצורה מקבילית, כך שבהינתן אותות הבקרה המתקבלים מיחידת הבקרה, מתבצעות הפעולות הנדרשות עבור מחזור הביצוע הנוכחי של הפקודה.

להלן המערכת:

### 3. Controller-based system:

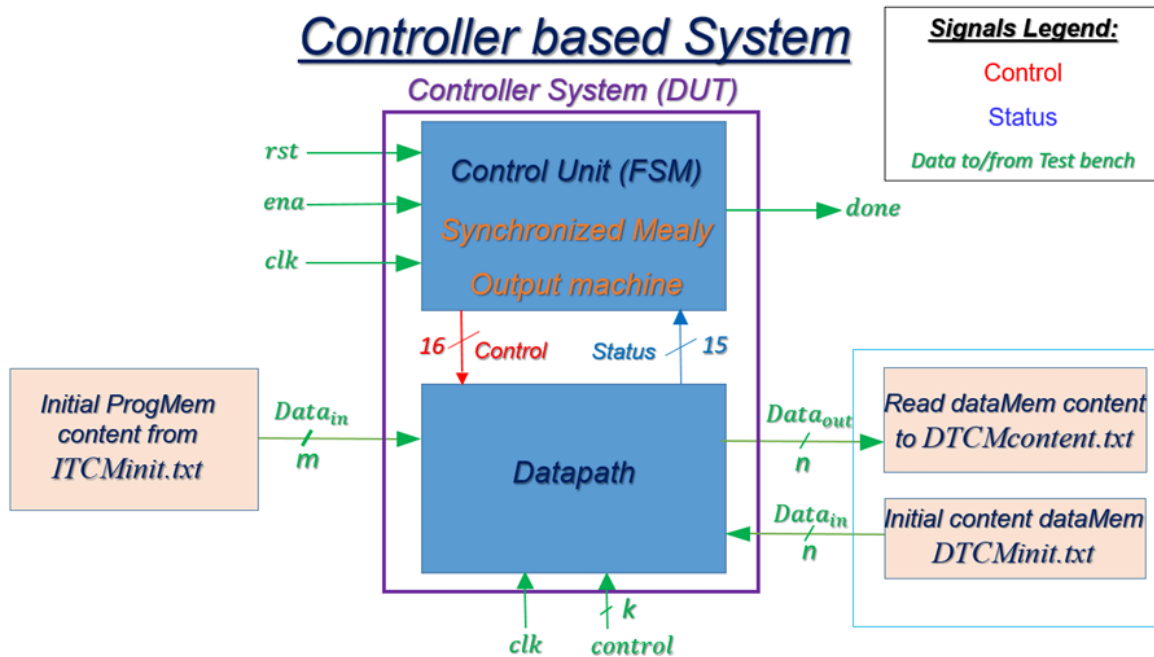


Figure 1: Overall DUT structure

המערכת מקבלת שני קבצי טקסט לצורך אתחול הזיכרונות:

- קובץ בשם ITCMinit.txt – מכיל את הוראות התכנית (instructions) שברצונו להריץ, ומשמש לאתחול זיכרון הפקודות (Program Memory / ITCM).
- קובץ בשם DTCMinit.txt – מכיל את הנתונים הראשוניים הנתונים לזיכרון הנתונים, לפי סדר כתובות הזיכרון, ומשמש לאתחול זיכרון הנתונים (Data Memory / DTCM).

בנוסף, המערכת מקבלת אותות בקרה הכוללים:

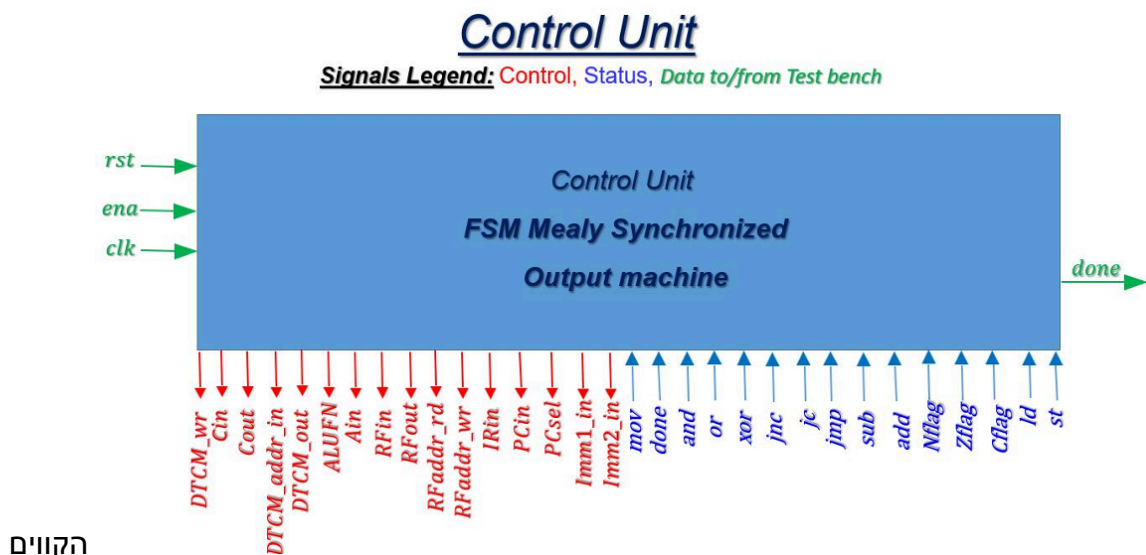
- clk – אות השעון של המערכת
- rst – אות איפוס המערכת
- ena – אות הפעלה המאפשר את ריצת המעבד

בסיום הרצת התוכנית, המערכת מעלה את הדגל done, המעיד על סיום הביצוע. לאחר מכן, תוכן זיכרון הנתונים (Data Memory / DTCM) נכתב לקובץ טקסט DTCMcontent.txt.

## Control Unit:

מהווה את הרכיב האחראי על ניהול רצף הביצוע של הפקודות במעבד ועל תיאום הפעולות בין רכיבי המערכת השונים. תפקידה המרכזי הוא לפרש את סוג הפקודה המתקבלת מה-Datapath, לעקוב אחר מצב הביצוע הנוכחי, ובהתאם לכך להפיק את אותות הבקרה הדרושים להפעלת רכיבי המעבד.

המימוש של יחידת הבקרה התבצע באמצעות מכונת מצבים סופית (FSM) מסוג Mealy המסונכרנת לאות השעון. בחירה זו מאפשרת לקביעת היציאות להתבסס הן על המצב הנוכחי של המכונה והן על הקלטים המתקבלים בזמן אמת, כגון סוג הפקודה ודגלי המצב (N, Z, C). יחידת הבקרה מקבלת מה-Datapath מידע לגבי הפקודה הנוכחית והסטטוס של פעולות קודמות, ובהתאם לכך מחליטה על מעבר בין המצבים השונים (כגון Fetch, Decode, ALU, Memory, WriteBack ו-Done), תוך הפקת אותות הבקרה המתאימים לכל שלב במחזור הביצוע של הפקודה.

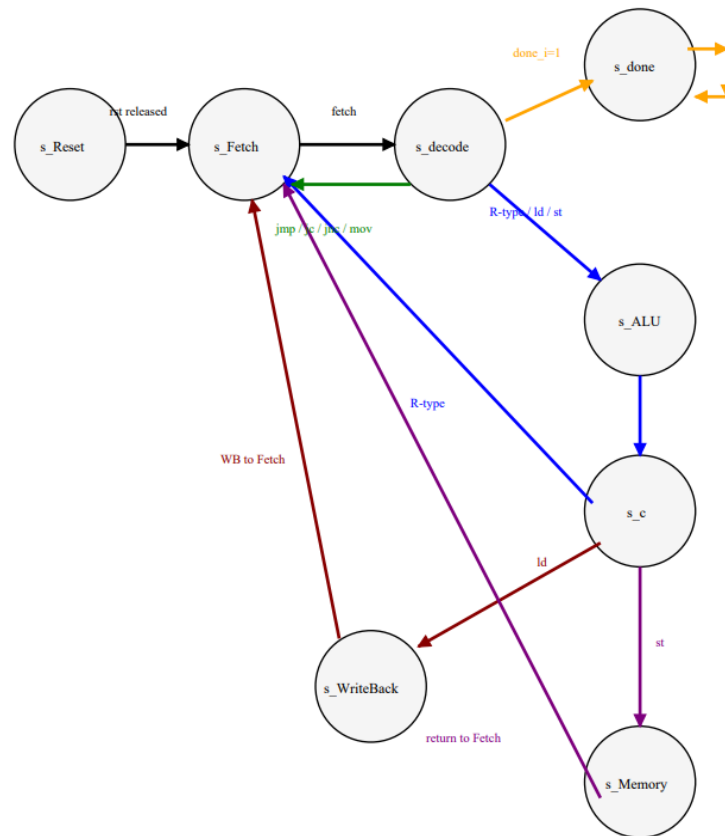


הקווים

האדומים הם היציאות מ-CONTROL אל ה-DATAPATH ואילו הכחולים הם הפוך. קווי ה-TB הם בצבע ירוק.

ה-FSM מכיל מספר סוגי פקודות בין היתר כתוצאה מהסוגים השונים של ההוראות המצורפות Rtype, Jtype, Itype

## להלן הFSM:

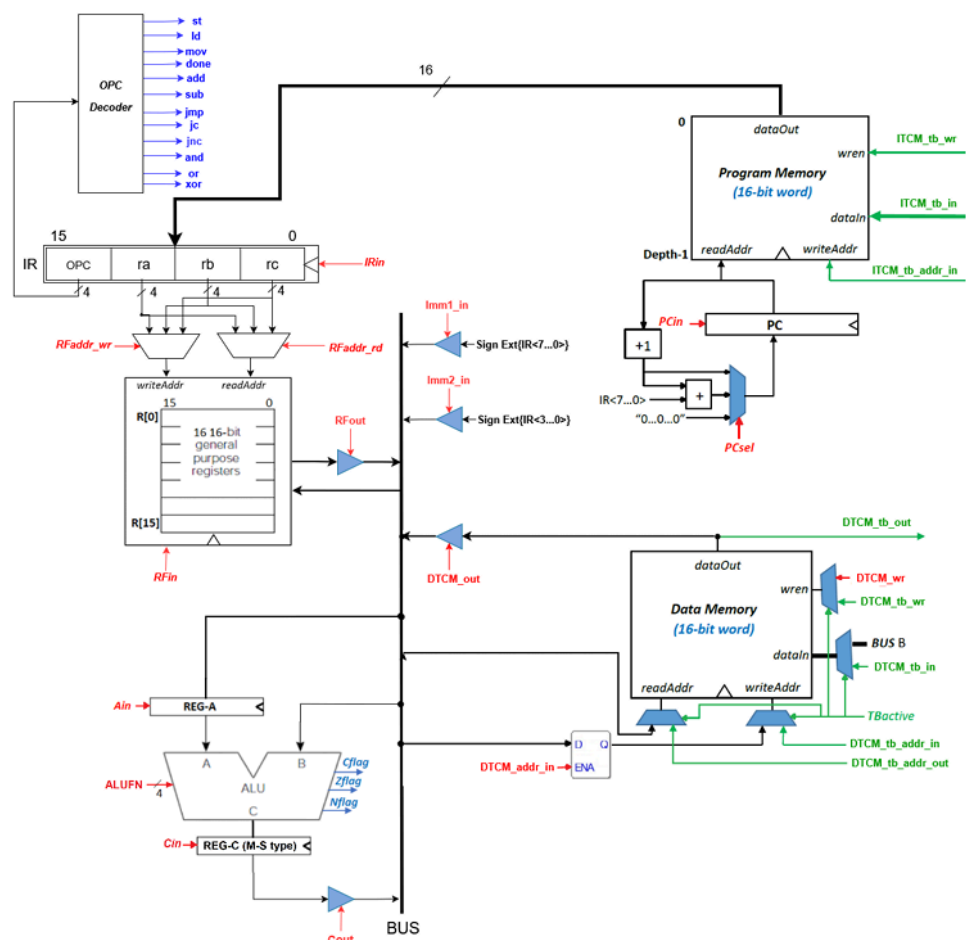


## להלן שמונה המצבים:

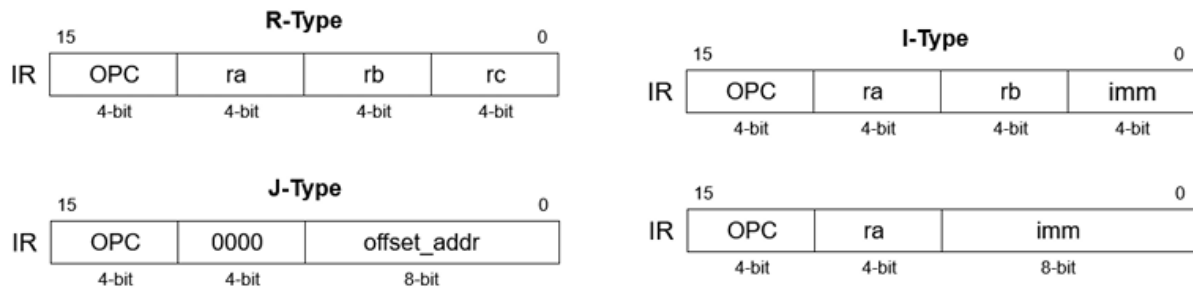
1. מצב Reset- מצב האתחול של המערכת, אליו מגיעים כאשר אות הבקרה rst עולה ל-1. במצב זה המערכת מאופסת ומוכנה להתחלת ריצת התוכנית.
2. מצב Fetch - במצב זה מחושבת כתובת הפקודה הבאה לביצוע באמצעות ה-Program Counter, ומתבצעת קריאת הפקודה המתאימה מזיכרון הפקודות.
3. מצב Decode- פיענוח הפקודה ב Datapath ושליחה ליחידת הבקרה ש-בהתאם לכך יודעת איזו הוראה לבצע וכך איזה דגלים לעלות.
4. מצב ALU- במצב זה מתבצע החישוב הנדרש ביחידת ה-ALU בהתאם לסוג הפקודה, תוצאת הפעולה נשמרת ברגיסטר C.
5. מצב C- במצב זה עבור פעולה מסוג Rtype כותבים ל- Register File וממשיכים למצב Fetch, עבור פעולת Ld, הולכים למצב Write Back ועבור פעולת st הולכים אל מצב Memory.
6. מצב Memory- במצב זה מתבצעת כתיבה לזיכרון הנתונים (Data Memory), ולאחר מכן חזרה למצב Fetch.
7. מצב Write Back- במצב זה מתבצעת כתיבת הנתון שהתקבל מזיכרון הנתונים אל ה-Register File, בהתאם לכתובת היעד של הפקודה.
8. מצב Done- צב סופי אינסופי, אליו מגיעים עם סיום ריצת התוכנית. במצב זה המערכת מעלה את אות done, אשר מאותת ל-Testbench כי ניתן לקרוא את תוכן הזיכרון הנתונים לצורך בדיקת תקינות.

### Data Path:

מהווה את החלק במעבד האחראי על ביצוע הפעולות בפועל בהתאם לאותות הבקרה המתקבלים מיחידת הבקרה. יחידה זו כוללת את הרכיבים המרכזיים של המעבד, כגון ה-ALU, ה-Register File, רגיסטרים פנימיים, ה-Program Counter, זיכרון הפקודות וזיכרון הנתונים. ממומש בארכיטקטורת Bus משותף עם שימוש ב-Tri-State Buffers, המאפשרים העברת מידע מבוקרת בין רכיבי המערכת השונים. בהתאם לאותות הבקרה, היחידה מבצעת פעולות כגון טעינת פקודות, קריאה וכתבייה לרגיסטרים, ביצוע חישובים אריתמטיים ולוגיים, גישה לזיכרון, ועדכון מונה הפקודות (PC) כחלק מתהליך הרצת התוכנית.



## פירוק הפעולות ב-IR:



כדי להמחיש את אופן פעולת ה-Datapath, נבחן את ביצוע הפקודה: `add ra, rb, rc`  
**מחזור 1 – Fetch:**

בשלב זה מתבצעת קריאת הפקודה מזיכרון הפקודות בהתאם לערך הנוכחי של ה-Program Counter.

$IR_{in} = 1$  → טעינת הפקודה ל-Instruction Register  
 $PC_{in} = 1$  → עדכון ה-PC לכתובת הבאה  
 $PC_{sel} = 00$  → בחירת  $PC + 1$   
 בסיום שלב זה הפקודה נמצאת ב-IR ומוכנה לפענוח.

## מחזור 2 – Decode:

בשלב זה מתבצע פענוח הפקודה.

ה-Opcode Decoder מזהה שמדובר בפקודת ADD, ויחידת הבקרה מגדירה את אותות הבקרה המתאימים.

במקביל:

קריאת rb מתוך Register File העלאתו ל-Bus טעינה ל-Register A שימוש באותות:

$RF_{out} = 1$   
 $RF_{addr\_rd} = 01$   
 $A_{in} = 1$

## מחזור 3 – ALU Execution:

בשלב זה:

rc נקרא מתוך Register File מועבר ל-Bus ה-ALU מקבל:

operand ראשון מ-Register A  
 operand שני מה-Bus  
 שימוש באותות:

$C_{in} = 1$   
 $RF_{out} = 1$   
 $RF_{addr\_rd} = 10$   
 $ALU_{FN} = ADD$

## מחזור 4 – Write Back:

בשלב האחרון:

תוצאת החישוב מ-Register C מועברת ל-Bus

נכתבת ל-Register File בכתובת ra.

שימוש באותות:

Cout = 1

RFin = 1

RFaddr\_wr = 10

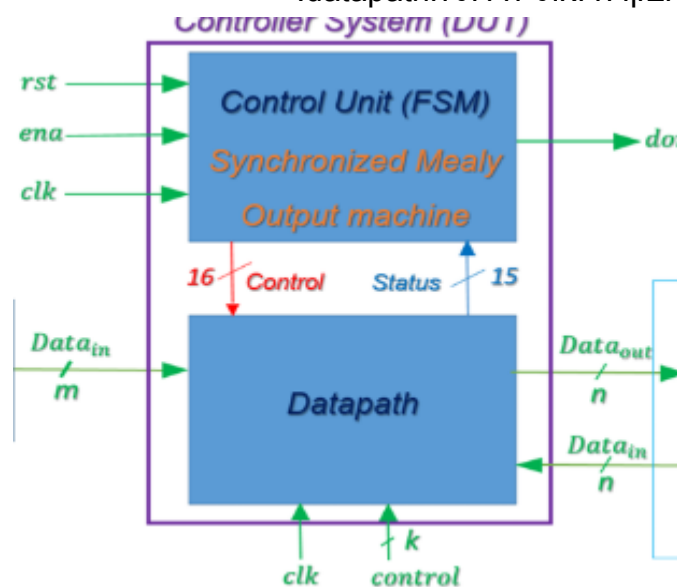
לאחר מכן חוזרים למצב Fetch.



## Top:

יחידת ה-TOP הינה היחידה העוטפת את המערכת כולה מקצה לקצה. ביחידה זו אנו מפעילים את שני המודולים המפעילים את המערכת כולה תחת מרכז שליטה אחד. תפקיד היחידה היא לבצע תוכניות שלמות על המעבד הכוללות שימוש מגוון ב-ISA. הרצת התוכנית למעבד כוללת שלושה חלקי פעולה. הראשון - לקרוא את הנתונים מקובץ בינארי ולטעון את התוכנית לזיכרון התוכנית ואת המידע לקובץ הזיכרון. לאחר מכן בחלק השני המעבד לוקח את הפיקוד ומבצע את התוכנית (כלומר בשלב זה  $TB\_ACTIVE = 0$ ) ולאחר סיום הרצה (המעבד מזהה את OPCODE של הפעולה DONE הנכללת ב-ISA) אנו עוברים לשלב האחרון שהוא לכתוב את הפלט לקובץ.

יחידת ה-TOP מתוארת ע"י התמונה הבאה - המעטפת הסגולה וסינגלי TB מזינים את היחידת הבקרה ואת יחידת datapath.

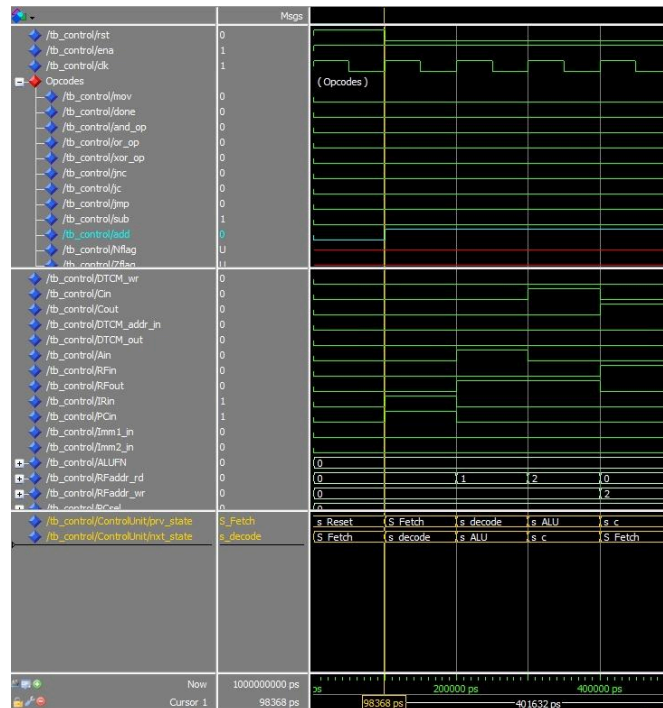


## תוצאות הסימולציה:

נחלק את תוצאות הסימולציה לשלושה: סימולציה עבור ה- Control, סימולציה עבור ה- Datapath וסימולציה עבור המערכת כולה.

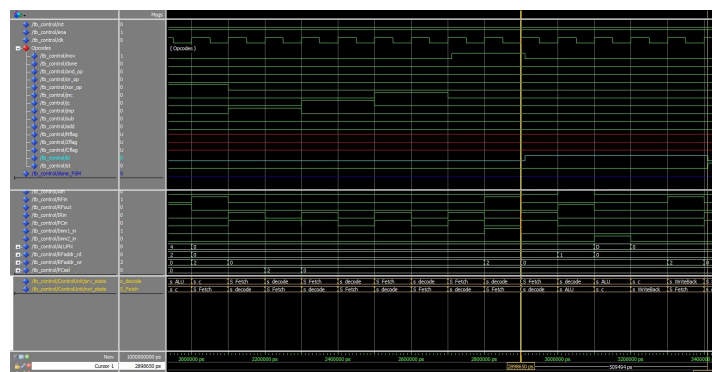
### סימולציה עבור ה- Control:

במהלך סימולציה זו נראה את אופן מעבר המצבים של מספר פעולות וכמות המחזורים הנדרשת לצורך ביצוע הפעולה.  
פעולת Add:



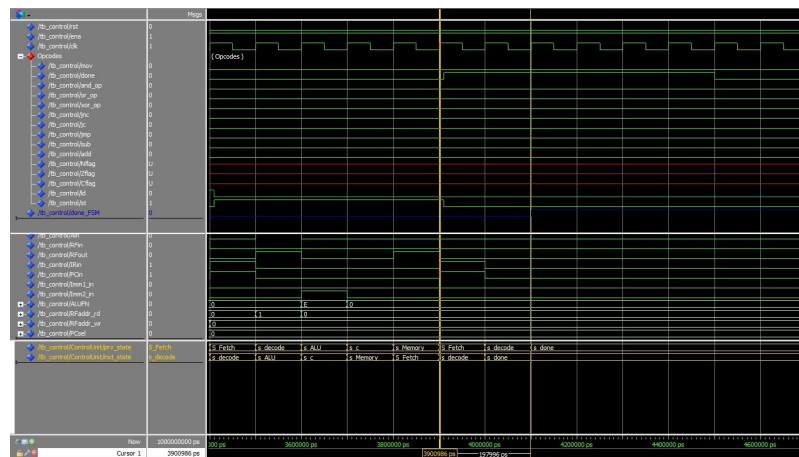
ניתן לשים לב כי מרגע עליית add ל-1 (מסומן בתכלת) נכנסים למצב Fetch ולוקח 4 מחזורי שעון עד סיום הפעולה, ניתן לשים לב לקווי הבקרה כמו למשל IRIN אשר עולה ל-1 בשלב ביצוע הFetch לצורך טעינת הIR.

### פעולת load:



ניתן לשים לב כי פעולה מסוג זה לוקחת 5 מחזורי שעון Fetch, Decode, ALU, C, Write Back ניתן לרשום למשל כי בשלב ה- WriteBack קו ה-RFIN עולה ל-1 מאחר וטוענים מידע אל ה- Register File.

פעולת done:

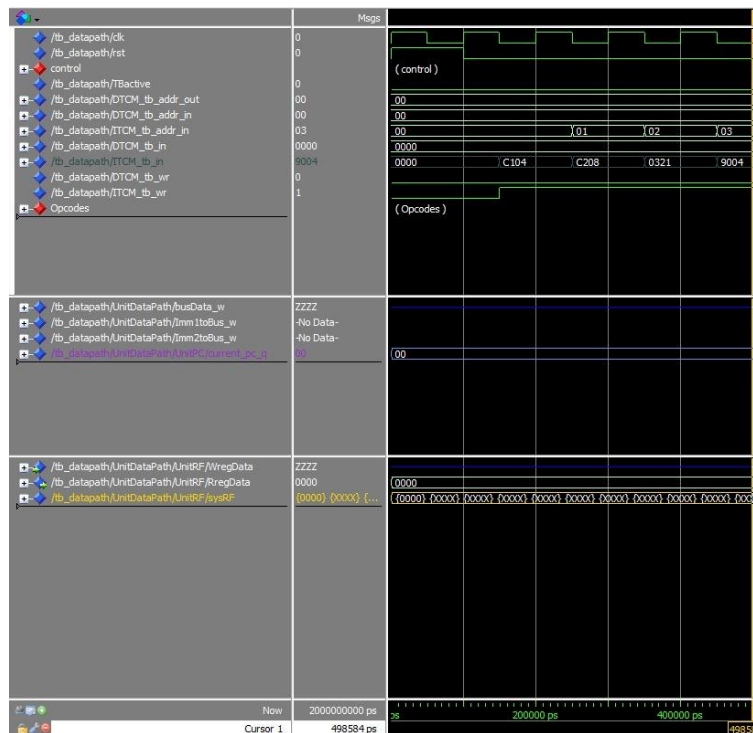


ניתן לשים לב כי פעולה זאת לוקחת למעשה 3 מחזורי שרון Fetch, Decode, Done .  
 ברגע Done זהו מצב סיום של המערכת ולכן קווי הבקרה אינם בשימוש.  
 אמנם הסימולציה מבוצעת עד ns 4500 אך בעיקרון מצב זה הוא אינסופי ונשארים בו עד אשר מבצעים ריסוס למערכת.

### סימולציה עבור ה-Data Path:

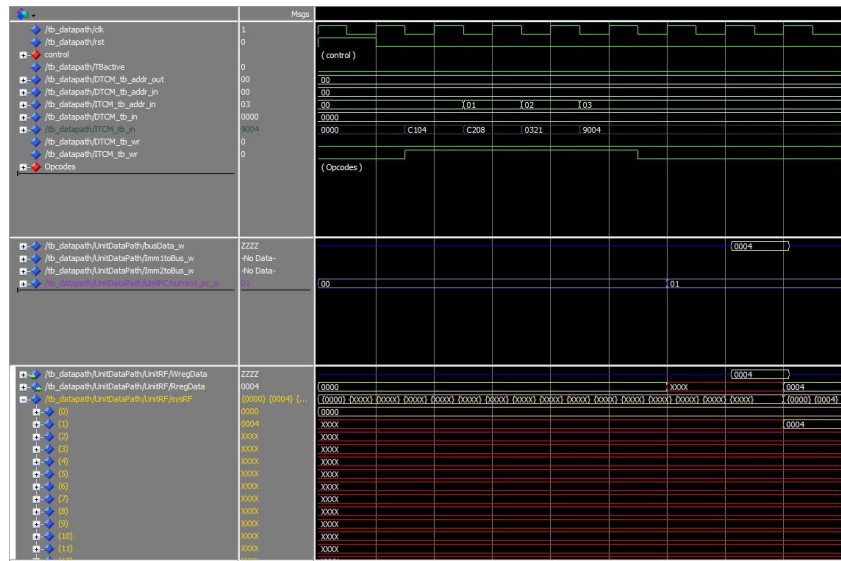
ביצוע הפקודות : `mov r1 4, mov r2, 8, add r3 r2 r1, jnc 4`

**שלב ראשון טעינת הפקודות לפי סדר ביצועתם:**

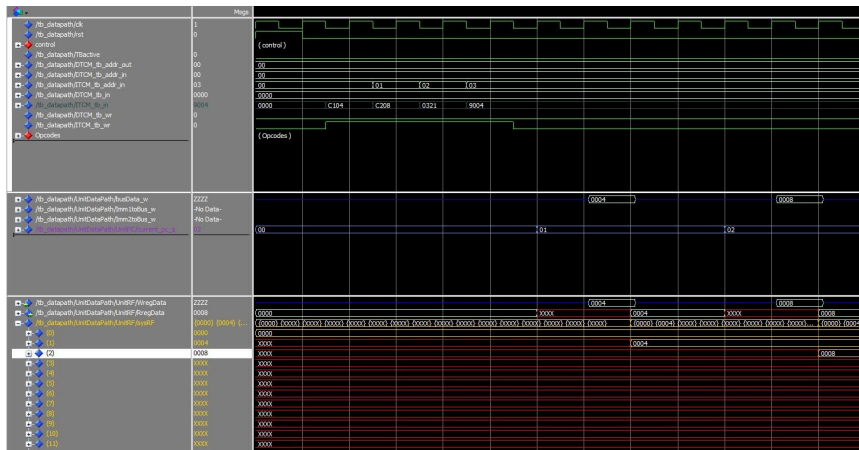


מקודד לפי ה-IR.

## שלב שני ביצוע פקודה 4, r1 מסר



ניתן לראות בRF כי בשורה מספר 1 נמצא הערך 4. בנוסף הPC מתקדם ב-1.  
שלב שלישי ביצוע פקודת 8 r2 מסר

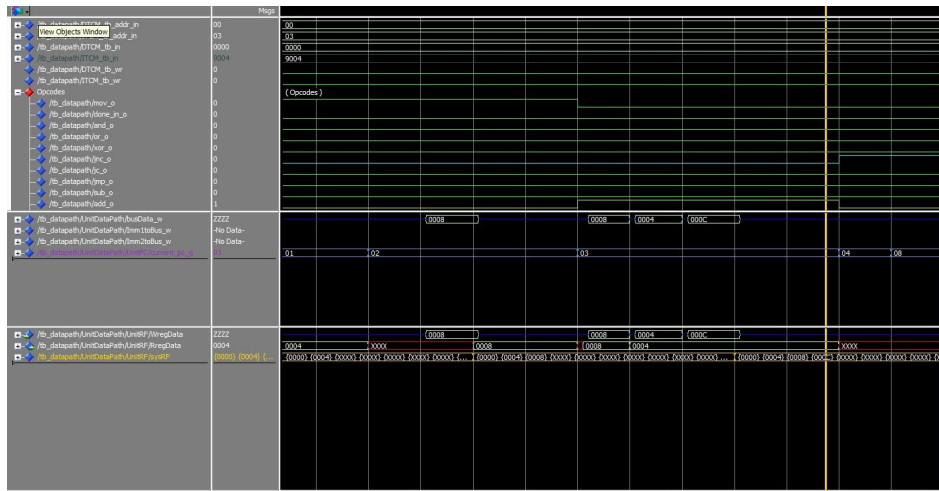


ניתן לראות בRF כי בשורה מספר 2 נמצא הערך 8. בנוסף הPC מתקדם ב-1.  
שלב רביעי ביצוע פקודת r3 r2 r1 add



ניתן לראות בRF כי בשורה מספר 3 נמצא הערך 12. בנוסף הPC מתקדם ב-1.

## שלב חמישי ביצוע פעולת 4 chn



ניתן לראות כי PC אכן עולה ב-4 מאחר וקו הבקרה chn עלה לצורך ביצוע הפעולה, מאחר ולא היה carry בפעולה האחרונה שבוצעה.

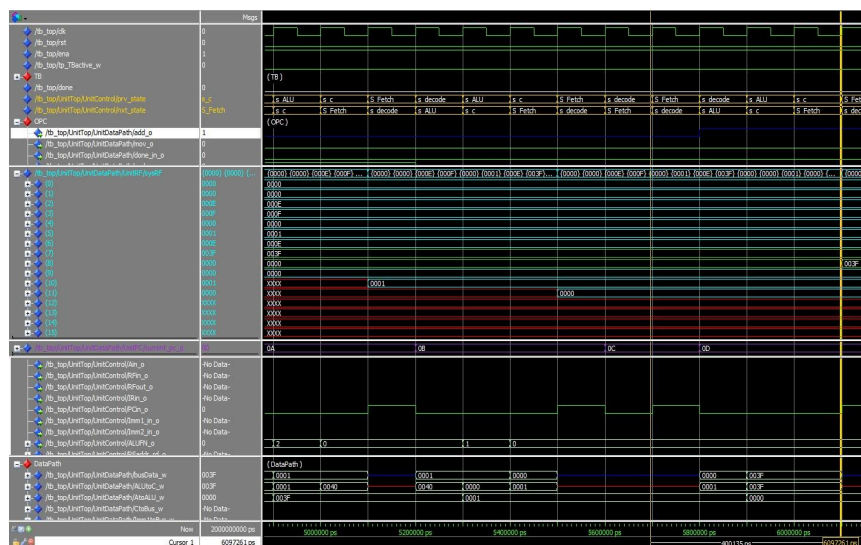
נציין בנוסף כי ניתן לראות לאורך כל הסימולציה את הערך על ה-Bus.

## סימולציה עבור המערכת כולה:

ב-סימולציה זו נרצה לאתחל את קבצי ה ttx המתאימים, כלומר את קובץ ה Instruction וקובץ אתחול

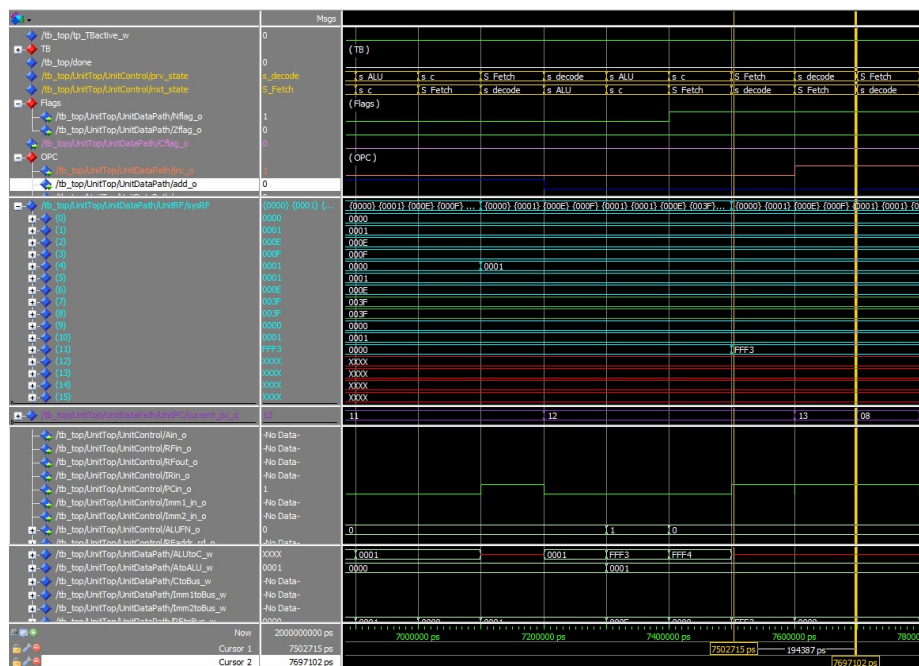
Data Mem. לאחר אתחול זה, נוכל להריץ סט פקודות בהתאם לקובץ ה פקודות שנכנסו מותאמות ISA. נבדוק האם הפקודות מתבצעות כראוי. אופן הבדיקה האם הבדיקות התבצעו כראוי הוא הן על ידי מבט על ההדפסות המתבצעות במהלך התוכנית והן על ידי בדיקת קובץ ה ttx של DataMem במוצא.

נדגים מתוך דיאגרת הגלים את אופן ביצוע הפעולה ADD R8,R8,R7



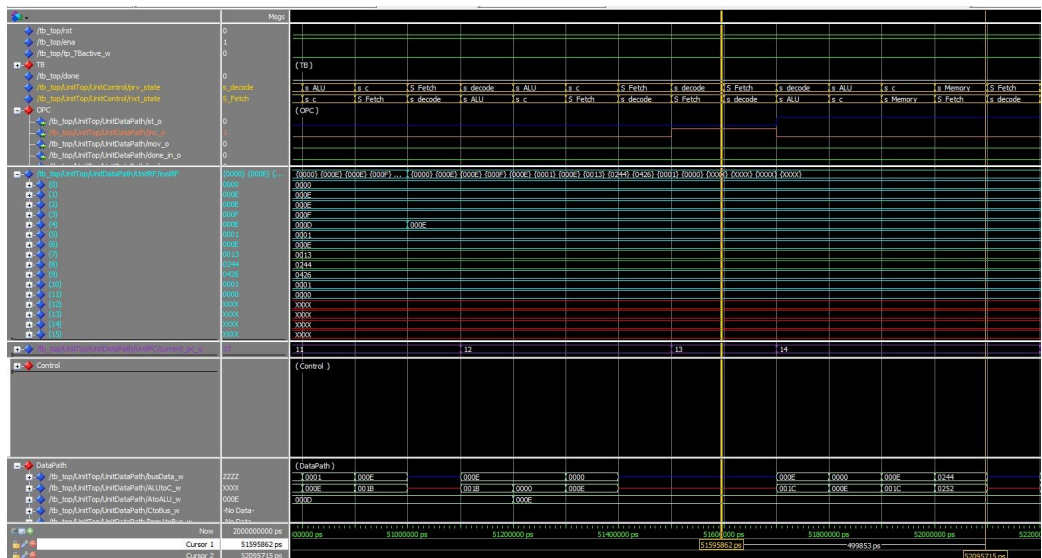
נתמקד במרווח בין שני הסמנים בצד ימין. נתן לראות ראשית שהפעולה לוקחת 4 מחזורי שעון כמצופה. נראה כי בשלב ה-`decode` הסינגל `add_s` עולה ל-1 לוגי כלומר אנו מזהים פעולת חיבור. ניתן לראות כי בתחילת הפעולה `R7 = 3F` ו-`R8 = 0`. בסיום הפעולה סינגל ה-`add_s` יורד ל-0 לוגי והערך `3F` נטען אל `R8` - כלומר פעולת החיבור עברה בהצלחה ואנו עוברים לפעולה הבאה. בנוסף ניתן לראות בקו הסגול את ה-`PC` שעולה ב-1 בכל פקודה (אלא אם זו פקודת הסתעפות) כנדרש.

נעבור לפקודה הבאה 11-JNC:



שוב נתמקד באיזור בין שני הסמנים. ראשית נשים לב שפקודת הסתעפות לוקחת 2 מחזורי שעון. בפקודה זו מה שמעניין אותנו זה להזיז את כתובת הPC קדימה או אחורה כנדרש בתוכנית אם דגל הCARRY שווה ל0. בורוד ניתן לראות שדגל הCARRY שווה ל0 ולכן JNC שווה ל1 ואנו מבצעים הסתעפות. ניתן לראות את כתובת הPC משתנה מ13 ל08 ובצורה דצימלית השינוי הוא מ19 ל8 כלומר מינוס 11 כנדרש. בעליית השעון הבאה המעבד יתחיל אכן מפקודה 8.

הפקודה האחרונה שנראה היא פקודת STORE - ST R8 0(R2)



נשים לב כי פקודה זו לוקחת כ-5 מחזורי שעון. נרצה בפקודה זו לאחסן את ערך R8 בכתובת אותה מחזיק הערך של R2. ניתן לראות כי  $R8 = 244$  ו  $R2 = 000E$  כלומר נרצה לאחסן בכתובת זו את 244. ניתן לראות כי הדבר אכן מצבע כצפוי - במחזור הרביעי בפקודה הBUS מעביר את הכתובת לטעינה ובמחזור החמישי הבאס מעביר את הערך הרצוי לתא הזיכרון ובעליית השעון הבאה הערך נשמר ומתחילה פקודה חדשה.

בנוסף מצורף הפלט של תוכניות מספר 5,6 אשר נמצא בקובץ DTCMcontent בכל אחת מתיקיות Output אשר בתיקיית EX5/6.  
פלט תוכנית מספר 6:

```
003F
021E
00F5
00BE
005B
0056
004E
0040
0053
0010
0018
003E
004F
0013
0244
0426
```

ניתן לשים לב כי המעבד פועל במדויק. 14 השורות הראשונות הינן ערכי המערך. השורה ה-15 הינה סכום ערכי המערך האי זוגיים והשורה האחרונה היא סכום ערכי המערך הזוגיים (בכתיב HEX).

פלט תוכנית מספר 5:

003F  
021E  
00F5  
00BE  
005B  
0056  
004E  
0040  
0053  
0010  
0018  
003E  
004F  
0013  
000D  
0138  
008D  
00A0  
005C  
0058  
0047  
003F  
003B  
000E  
002B  
000C  
0047  
005A  
0032  
0356  
0068  
015E  
FFFF  
00AE  
0007  
007F  
0018  
001E  
FFED  
004A  
0008  
006D